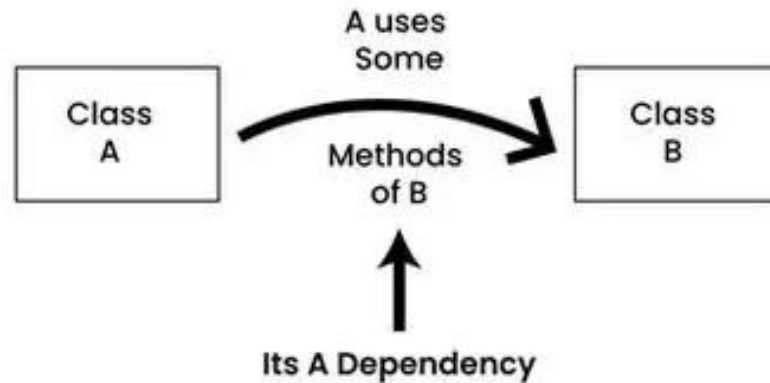




Dependency Injection (DI)

What is the Dependency Injection Design Pattern in Java?

*design pattern in Java is a powerful technique for managing dependencies between objects. It aims to decouple classes from their dependencies, making code more **flexible, testable, and maintainable**.*



Dependency Injection Design Pattern

How Dependency Injection Design Patterns in Java work?

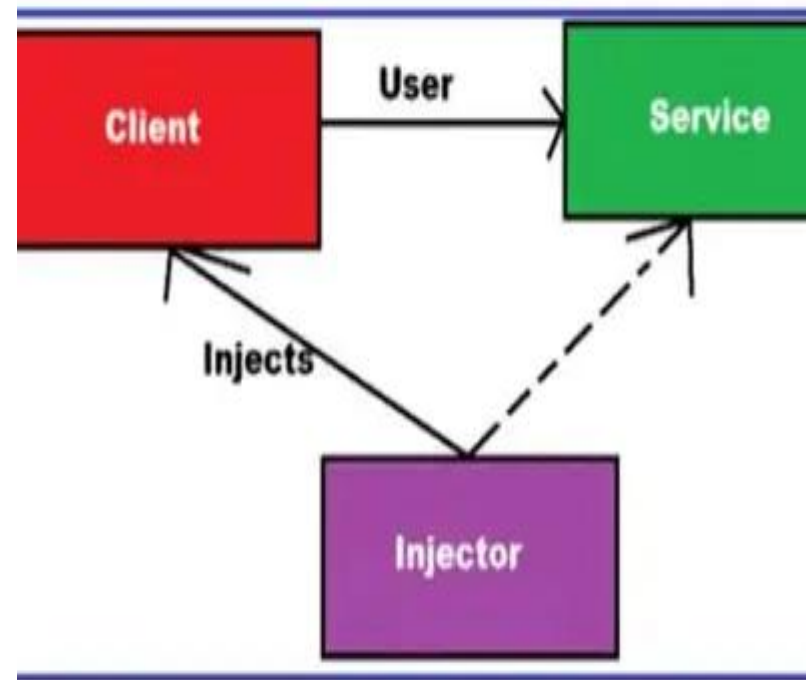
Identify dependencies:

- Analyze the classes in your application to determine which objects they rely on to function. These objects are their dependencies.
- Define clear interfaces or abstract classes to represent these dependencies, promoting loose coupling.

Inject dependencies:

Constructor Injection: Pass the dependencies as arguments to the class's constructor. This makes them mandatory for object creation and ensures immutability.

Setter Injection: Provide setter methods to set the dependencies after object creation. This offers more flexibility but can make the class mutable.



Types of DI in Spring

Constructor Injection

Pass the dependencies as arguments to the class's constructor. This makes them mandatory for object creation and ensures immutability.

```
@Component 2 usages
public class UserService {

    private final NotificationService notificationService; 2 usages

    public UserService(NotificationService notificationService) {
        this.notificationService = notificationService;
    }

    public void registerUser() { 1 usage
        notificationService.sendNotification("User Registered");
    }
}
```

Setter Injection

Provide setter methods to set the dependencies after object creation. This offers more flexibility but can make the class mutable.

```
@Component 2 usages
public class UserService {

    private EmailService emailService; 2 usages
    @Autowired
    public UserService(EmailService emailService) {
        this.emailService = emailService;
    }

    public void sendEmail(String to, String subject, String text) { 1 usage
        emailService.sendEmail();
    }

}
```

Comparison Table

Feature	Constructor Injection	Setter Injection
Dependency Requirement	Mandatory	Optional
Immutability	Promotes immutable objects	Mutable objects allowed
Readability	Clear at object creation	Less explicit
Circular Dependency	Not suitable	Can handle circular deps
Testing	Easier to test	Slightly harder

When to Use Which

•Constructor Injection:

- When dependencies are **mandatory**
- When you want **immutable objects**
- Preferred in **modern Spring development**

•Setter Injection:

- When dependencies are **optional**
- When you need **flexibility** to change dependencies later